

Phase 1 Implementation Plan — Validation & Dashboard

Project: Fintech Sentiment Intelligence Analysis

Author: John Kirima

Goal: Elevate the project from 84% → 95% (“top 1% portfolio”) by closing the two gaps flagged in the brutal review:

1. **No validated ground truth** — the severity engine has never been measured against human labels (no F1/precision/recall).
2. **No interactive dashboard** — findings live only in static notebooks and PNGs.

Phase 1 delivers both. Phase 2 (full case-study page on the website) comes later.

0. Current State (verified from repo audit)

Notebooks (`notebooks/`)

Notebook	Role
<code>01_scraping.ipynb</code>	Google Play scraping → <code>data/raw/</code>
<code>02_eda.ipynb</code>	Cleaning + feature engineering → <code>data/clean/</code> (10,386 rows, 16 cols)
<code>03_sentiment_nlp.ipynb</code>	VADER compound scoring
<code>04_Evaluation + mismatch analysis.ipynb</code>	VADER vs star-rating classification report + confusion matrix
<code>05_hidden_negative_audit.ipynb</code>	Severity Engine defined here + hidden-negative audit
<code>06_competitive intelligence and Benchmarking audit.ipynb</code>	Per-app hidden-negative benchmarking + gap heatmap
<code>07_topic_modelling.ipynb</code>	BERTopic on moderate→critical hidden negatives
<code>08_executive_summary.ipynb</code>	(empty placeholder)

Data

- `data/clean/all_apps_clean.csv` — 10,386 reviews, 16 columns. Key cols: `app_name`, `review_id`, `rating`, `review_text`, `review_clean`, `is_negative`, `rating_tier`.
- `data/clean/negative_reviews.csv` — 3,177 reviews (rating ≤ 2).
- App split: Cash App 2,600 / Venmo 2,597 / Chime 2,595 / PayPal 2,594.

- Rating split: 1★ 2,763 · 2★ 414 · 3★ 402 · 4★ 662 · 5★ 6,145.

Existing severity framework (in notebooks 05/06/07 — currently duplicated inline in each)

The **Fintech Severity Engine** is a rule-based, keyword-lexicon classifier:

```
severity_keywords = {
    5: ['locked out', 'account locked', 'account closed', 'money missing', 'funds missing',
        'stolen', 'fraud', 'scam', 'cannot access', 'lost my money', 'money gone', 'unauthorized'],
    4: ['declined', 'card declined', 'transfer failed', 'verification failed', 'failed', 'frozen',
        'pending', 'verification', "can't send", "can't receive", 'not working', 'blocked'],
    3: ['customer service', 'support', 'refund', 'delay', 'late', 'problem', 'issue', 'error', 'wrong', 'charge'],
    2: ['slow', 'bug', 'glitch', 'annoying', 'confusing', 'difficult'],
    1: ['minor', 'small', 'slight', 'okay', 'fine']
}
# First match wins, scanning level 5 → 1. Default = 1 (Low).
severity_map = {1: 'Low', 2: 'Low-Moderate', 3: 'Moderate', 4: 'High', 5: 'Critical'}
```

Hidden negative = `rating_sentiment == 'Negative' AND vader_sentiment != 'Negative'` (a genuinely negative review VADER failed to catch).

⚠ **Key weakness this plan fixes:** the severity engine has never been validated. We assert “26% miss rate” and “\$2-4M opportunity” but have no human-labeled ground truth proving the severity scores are correct. Phase 1 produces that proof.

Not yet built

- `dashboard/` contains only an empty `screenshots/` folder — **no Streamlit app exists**.
- No `data/labels/` folder.
- No `outputs/tables/` or `outputs/exports/` content (empty).
- Severity keyword dict is copy-pasted across 3 notebooks — should be extracted to a shared module (`src/severity.py`) so labeling, validation, and dashboard all use one source of truth.

1. Manual Labeling Setup

1.1 Objective

Produce a human-labeled gold set of **200 reviews** with two labels each, so we can validate the automated engines:

- `true_sentiment` ∈ {Negative, Neutral, Positive} — validates VADER + the “hidden negative” claim.
- `true_severity` ∈ {1,2,3,4,5} — validates the Severity Engine (the headline contribution).

1.2 Sampling strategy (stratified, reproducible)

A pure random sample of 200 would be ~59% 5★ reviews and contain almost no Critical-severity cases — useless for validating the severity engine. We stratify to guarantee coverage of the cells that matter.

Stratification dimensions:

1. **App** (4) — equal representation so per-app benchmarking claims are defensible.
2. **Rating tier** — Negative (1-2★) / Neutral (3★) / Positive (4-5★).
3. **Predicted severity** — oversample predicted High(4)/Critical(5) reviews, since those drive the business case and are rarest.

Target allocation (200 reviews):

Stratum	Target n	Rationale
Per app	50 each (×4)	Equal footing for competitive claims
Negative-rating reviews	~120 (60%)	Where severity & hidden-negatives live
Neutral-rating reviews	~30 (15%)	Boundary cases VADER struggles with
Positive-rating reviews	~50 (25%)	Confirm engine doesn't over-flag happy users
Predicted Critical/High (sev 4-5)	≥ 60 total	Guarantee enough positives to compute severity recall
Hidden-negative pool	≥ 40	Directly validate the flagship "VADER missed it" claim

Implementation: notebooks/09_sampling_for_labeling.ipynb (or scripts/build_label_sample.py)

- Load `all_apps_clean.csv` , apply the shared severity + VADER functions.
- Use `pandas.groupby(...).sample(n=..., random_state=42)` per stratum; dedupe on `review_id` ; top up to exactly 200.
- **Fix the random seed (42)** for full reproducibility.
- Export the unlabeled sheet to `data/labels/label_queue.csv` with columns:
`review_id, app_name, rating, review_text, predicted_vader_sentiment, predicted_severity, true_sentiment(blank), true_severity(blank), notes(blank)` .
- **Blind the labeler:** keep predicted columns in a separate file (`label_queue_blind.csv` without prediction cols) so the human isn't anchored by the model's guess. Merge predictions back only at scoring time.

1.3 Labeling interface

Keep it lightweight — 200 rows is a 1-2 hour manual task, no need for Label Studio.

Recommended: a small Streamlit labeling helper `dashboard/label_tool.py` (reused Streamlit skill from the main dashboard):

- Shows one review at a time: full text, app, rating.
- Two button rows: sentiment (Neg/Neu/Pos) and severity (1-5) with tooltip definitions.
- "Skip" + free-text notes; autosaves to `data/labels/labels.csv` after every entry; resumes where you left off.
- Progress bar (`x / 200`).

Fallback (zero-code): the exported `label_queue.csv` opens directly in Excel/Google Sheets — fill `true_sentiment` and `true_severity` columns by hand. This is the fastest path if you just want to grind through them.

Severity rubric (must be written into the tool + this doc so labeling is consistent):

Score	Label	Definition	Example
5	Critical	Money lost/inaccessible, account locked/closed, fraud	“account locked with \$2k inside, can’t reach anyone”
4	High	Core function broken: transfer/verification/card failure	“transfer failed 3 times, card declined everywhere”
3	Moderate	Friction, needs support, refund/delay disputes	“waiting a week for support to fix a wrong charge”
2	Low-Moderate	Annoyance, bugs, UX complaints	“app is slow and buggy after update”
1	Low	Minor/neutral/positive, no operational failure	“works fine, wish it had dark mode”

1.4 Storage

```
data/labels/
├── label_queue.csv          # 200 sampled rows + predictions (for scoring)
├── label_queue_blind.csv    # same rows, prediction cols hidden (for labeling)
├── labels.csv              # human labels: review_id, true_sentiment, true_severity,
notes
├── labeling_rubric.md      # the severity rubric above, frozen for consistency
```

Commit all four to git so the ground truth is auditable and reproducible.

2. Validation Metrics

2.1 New notebook: `notebooks/10_validation_metrics.ipynb`

Joins `data/labels/labels.csv` back to predictions on `review_id` and computes formal metrics.

2.2 Severity Engine validation (the headline result)

Treat severity as (a) multi-class and (b) a binary “high-severity” detector.

Multi-class (1-5):

```
from sklearn.metrics import classification_report, confusion_matrix, cohen_kappa_score
print(classification_report(y_true_sev, y_pred_sev, digits=3))
# precision/recall/F1 per class + macro/weighted
print("Cohen's kappa:", cohen_kappa_score(y_true_sev, y_pred_sev))
# agreement beyond chance
```

- Report **macro-F1** (treats rare Critical class fairly) and **weighted-F1**.
- Plot the 5×5 confusion matrix → save `outputs/charts/severity_confusion_matrix.png`.

Binary high-severity detector (the business-critical framing):

Collapse to `is_high_severity = severity >= 4`. This is what the \$2-4M claim rests on — “can the engine reliably flag the expensive complaints?”

```
precision, recall, f1, _ = precision_recall_fscore_support(
    y_true_high, y_pred_high, average='binary')
```

- Report **precision** (of flagged-high, how many truly high — false-alarm cost) and **recall** (of truly-high, how many caught — miss cost). Both matter for the exec narrative.

2.3 VADER validation (supports the “26% miss” claim)

```
print(classification_report(true_sentiment, vader_sentiment, digits=3))
```

- Confirm VADER’s recall on Negative class on the human-labeled set — this is the honest, defensible version of the “59% recall / 26% miss” numbers currently derived from star ratings alone.

2.4 Deliverables from validation

- `outputs/tables/severity_validation_report.csv` — per-class precision/recall/F1.
- `outputs/tables/vader_validation_report.csv`.
- `outputs/charts/severity_confusion_matrix.png`.
- A markdown summary block: **“Severity Engine: macro-F1 = 0.XX, high-severity recall = 0.XX on 200 human-labeled reviews”** — this single sentence is what upgrades the README/case study from asserted to validated.
- **Honesty guardrail:** report the real numbers even if modest. A validated F1 of 0.72 with a documented improvement path beats an unvalidated “trust me.” If scores are low, add a short “limitations & next steps” note (e.g., regex/negation handling, expand lexicon).

3. Dashboard Architecture

3.1 Stack

- **Streamlit** multi-page app (matches the `dashboard/` folder already scaffolded; check `/home/ubuntu/skills/` for a Streamlit skill before building).
- **Brutalist / terminal aesthetic** to match johnkirima.com: monospace font (Space Mono / IBM Plex Mono), near-black background `#0a0a0a`, high-contrast text, sharp borders, minimal color except severity reds/oranges. Set via `.streamlit/config.toml` theme + light custom CSS.
- Reads the existing CSVs directly (no DB needed at this scale). Cache with `@st.cache_data`.

3.2 Shared logic module (build FIRST)

Extract the duplicated severity/VADER code into `src/analysis.py` :

```
# src/analysis.py
def get_vader_score(text) -> float
def vader_label(compound) -> str
def rating_label(rating) -> str
def compute_severity(text) -> int
SEVERITY_KEYWORDS, SEVERITY_MAP
def score_dataframe(df) -> df # adds all derived columns in one call
```

Notebooks 05/06/07, the labeling sampler, the validator, and the dashboard all import from here → one source of truth, no drift.

3.3 App structure

```

dashboard/
├── app.py                # landing / overview page
├── pages/
│   ├── 1_Competitive_Analysis.py
│   ├── 2_Topic_Explorer.py
│   └── 3_Model_Validation.py
├── label_tool.py        # standalone labeling helper (section 1.3)
├── .streamlit/config.toml # brutalist theme
└── screenshots/        # existing (for README embeds)

```

3.4 Views

Page 0 — Overview (app.py)

- KPI cards (terminal-style): total reviews (10,386), 4 apps, hidden-negative rate, Critical+High miss count, validated severity F1.
- One-paragraph business framing + the \$2-4M recovery estimate.
- Rating distribution + weekly negative-volume charts (reuse EDA logic).
- Global filters in sidebar: app multiselect, rating range, severity range, date range → propagate to all pages via `st.session_state`.

Page 1 — Competitive Analysis

- The benchmarking table from notebook 06 (Hidden Neg Rate, Avg Hidden Severity, Critical/High misses per app), sortable.
- Competitive gap heatmap (app × severity) — interactive (Plotly) version of `competitive_gap_heatmap.png`.
- Bar charts: hidden-neg rate by app; Critical+High misses by app. Headline: Venmo ≈ 2× Cash App negative rate.

Page 2 — Topic Explorer

- BERTopic themes from notebook 07 (App Performance & Support, PayPal Disputes, Venmo Friction, Chime Banking, AI/Bot Frustration, Fund Holds/Card, Fraud/Scam).
- Theme × app stacked bars + avg-severity-per-theme.
- **Drill-down:** pick a theme+app → table of actual review texts (with rating, severity, VADER label) so a recruiter sees the raw evidence behind each number.

Page 3 — Model Validation (the differentiator)

- Displays the Phase-2 metrics: severity confusion matrix, per-class F1 table, high-severity precision/recall, VADER recall on the labeled set.
- Side-by-side “VADER said Neutral / Human said Critical” example gallery — makes the hidden-negative problem visceral.
- States sample size (n=200), stratification method, and seed → signals rigor.

3.5 Data sources

Source	Used by
<code>data/clean/all_apps_clean.csv</code>	all pages (scored live via <code>src/analysis.py</code> , cached)
<code>data/labels/labels.csv</code>	Model Validation page
<code>outputs/tables/*validation_report.csv</code>	Model Validation page
<code>outputs/charts/*.png</code>	fallback static embeds

Performance note: scoring 10,386 rows with VADER on every app load is fine once, but pre-compute once and cache. Optionally persist a scored parquet to `outputs/exports/scored_reviews.parquet` and have the dashboard read that.

3.6 Deployment

Local (`streamlit run dashboard/app.py`) for development + screenshots. For a live link, **Streamlit Community Cloud** (free, connects to the GitHub repo) is the lowest-friction option; link it from the case-study page in Phase 2. Capture screenshots into `dashboard/screenshots/` for the README regardless.

4. Execution Order (recommended)

1. **Extract** `src/analysis.py` — de-duplicate severity/VADER logic; refactor notebooks 05-07 to import it. (unblocks everything)
2. **Build sampler** → `data/labels/label_queue*.csv` (200 stratified, seed=42).
3. **Label** the 200 reviews (Streamlit `label_tool.py` or Sheets) → `data/labels/labels.csv` .
4. `10_validation_metrics.ipynb` → severity + VADER F1/precision/recall, confusion matrix, saved tables/charts.
5. **Build Streamlit dashboard** (overview → competitive → topics → validation).
6. **Update README** with the validated F1 headline + dashboard screenshots/link.
7. **Commit** each step to git (mandatory version control).

5. Definition of Done (Phase 1)

- [] `data/labels/labels.csv` with 200 human-labeled reviews committed.
- [] Severity engine has a reported **macro-F1 and high-severity recall** on the labeled set.
- [] VADER negative-recall reported on the human-labeled set (replaces star-rating-only estimate).
- [] Streamlit dashboard runs locally with 4 pages, brutalist theme, live filters.
- [] README updated: “validated” replaces “asserted”; dashboard screenshots + (optional) live link added.
- [] `src/analysis.py` is the single source of truth for scoring logic.